MongoDB Complete Commands – Student Collection

Student ID	2520090170
Department	Computer Science and Information Technology (CSIT)
Course	Database Management Systems (DBMS)

Aim

To demonstrate MongoDB database and collection creation, insertion of student documents, and execution of CRUD operations (Create, Read, Update, Delete). Additionally, this practical covers comparison operators, logical operators, sorting, limiting, projection, indexing, distinct values, and aggregation framework operations on the student collection using MongoDB Compass and mongosh.

Software Required

- MongoDB Community Edition
- MongoDB Compass (GUI)
- mongosh (MongoDB Shell / Command Line Interface)

Procedure

1. Open MongoDB Compass and connect to the local MongoDB server.
2. Open the embedded MongoDB shell (mongosh) within Compass.
3. Create or select the database "studentDB".
4. Create the "students" collection within studentDB.
5. Insert the supplied student documents using insertMany() and insertOne() operations.
6. Execute MongoDB query commands including find(), findOne(), countDocuments(), and updateOne().
7. Apply comparison operators ($gt, $gte, $ne) and logical operators ($and, $or).
8. Perform update operations with $set and $rename operators.
9. Execute delete operations using deleteOne() and deleteMany().
10. Apply sorting, limiting, and skip operations.
11. Demonstrate projection to select specific fields.
12. Create indexes on collection fields.
13. Execute aggregation pipeline operations including $match, $group, $sum, $max, and $project.

Commands and Outputs

1. Database, Collection and Data Insertion

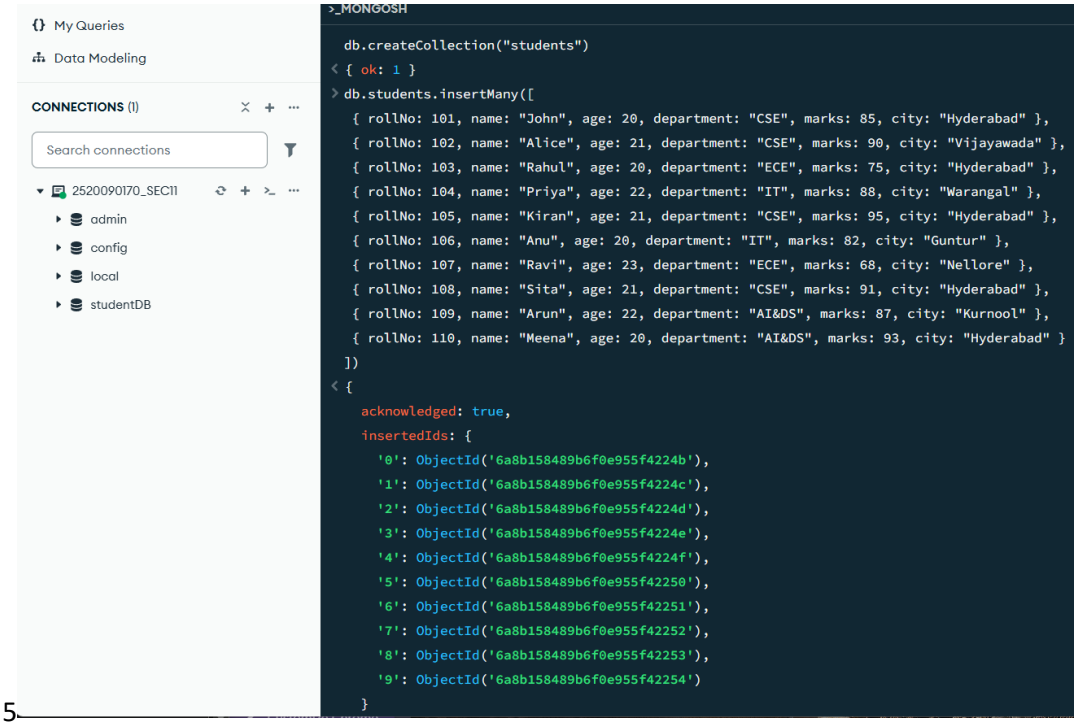
Creating the Database and Collection

```
db.createCollection("students")
```

Inserting Multiple Student Documents (insertMany)

```
db.students.insertMany([...])
```

5.



The createCollection() command creates a new collection named 'students'. The insertMany() operation inserts 10 student documents (rollNo 101–110) with fields: rollNo, name, age, department, marks, and city. The response shows all inserted document IDs.

Inserting a Single Student Document

```
db.students.insertOne({ rollNo: 111, name: "Vijay", age: 21, department: "CSE", marks: 86, city: "Hyderabad" })
```



The insertOne() operation adds a single new student (rollNo 111). The output confirms successful insertion with acknowledged: true and the inserted document ID.

2. Read Operations

Finding a Single Document

```
db.students.findOne({ rollNo: 101 })
```

The findOne() method retrieves the first student document matching rollNo 101, displaying all fields including the MongoDB-generated _id.

Counting Documents

```
db.students.countDocuments({ department: "CSE" })
```

The countDocuments() method returns the total number of CSE students. The count is 5, indicating five CSE department students in the collection.

3. Comparison Operators

Finding Documents with $ne (Not Equal)

```
db.students.find({ department: { $ne: "CSE" } })
```

```
> db.students.find({ department: { $ne: "CSE" } })
< {
    _id: ObjectId('6a8b158489b6f0e955f4224d'),
    rollNo: 103,
    name: 'Rahul',
    age: 20,
    department: 'ECE',
    marks: 75,
    city: 'Hyderabad'
  }
  {
    _id: ObjectId('6a8b158489b6f0e955f4224e'),
    rollNo: 104,
    name: 'Priya',
    age: 22,
    department: 'IT',
    marks: 88,
    city: 'Warangal'
  }
  {
    _id: ObjectId('6a8b158489b6f0e955f42250'),
    rollNo: 106,
    name: 'Anu',
    age: 20,
    department: 'IT',
    marks: 82,
    city: 'Guntur'
  }
```

The $ne (not equal) operator retrieves all student documents whose department is not 'CSE'. Results show students from ECE, IT, and AI&DS departments.

Finding Documents with $and (Logical AND)

```
db.students.find({ $and: [{ department: "CSE" }, { marks: { $gt: 85 } }] })
```

```
> db.students.find({
    $and: [
      { department: "CSE" },
      { marks: { $gt: 85 } }
    ]
  })
< {
    _id: ObjectId('6a8b158489b6f0e955f4224c'),
    rollNo: 102,
    name: 'Alice',
    age: 21,
    department: 'CSE',
    marks: 90,
    city: 'Vijayawada'
  }
  {
    _id: ObjectId('6a8b158489b6f0e955f4224f'),
    rollNo: 105,
    name: 'Kiran',
    age: 21,
    department: 'CSE',
    marks: 95,
    city: 'Hyderabad'
  }
```

The $and operator combines two conditions: students must be from CSE AND have marks > 85. Results show Alice (90 marks) and Kiran (95 marks), both CSE students meeting the marks criteria.

4. Distinct Values

Finding Distinct Department Values

```
db.students.distinct("department")
```



The distinct() method returns all unique department values: AI&DS, CSE, ECE, and IT.
The second query retrieves unique departments for students with marks greater than or equal to 80.

5. Update Operations

Updating a Single Document with $set

```
db.students.updateOne({ rollNo: 101 }, { $set: { marks: 92 } })
```



The updateOne() method with $set operator modifies the marks of student rollNo 101 from 85 to 92. Response shows matchedCount: 1 and modifiedCount: 1, confirming successful update.

Updating Multiple Documents with $set

```
db.students.updateMany({ department: "CSE" }, { $set: { status: "Active" } })
```

```
db.students.updateMany(
 { department: "CSE" },
 { $set: { status: "Active" } }
)

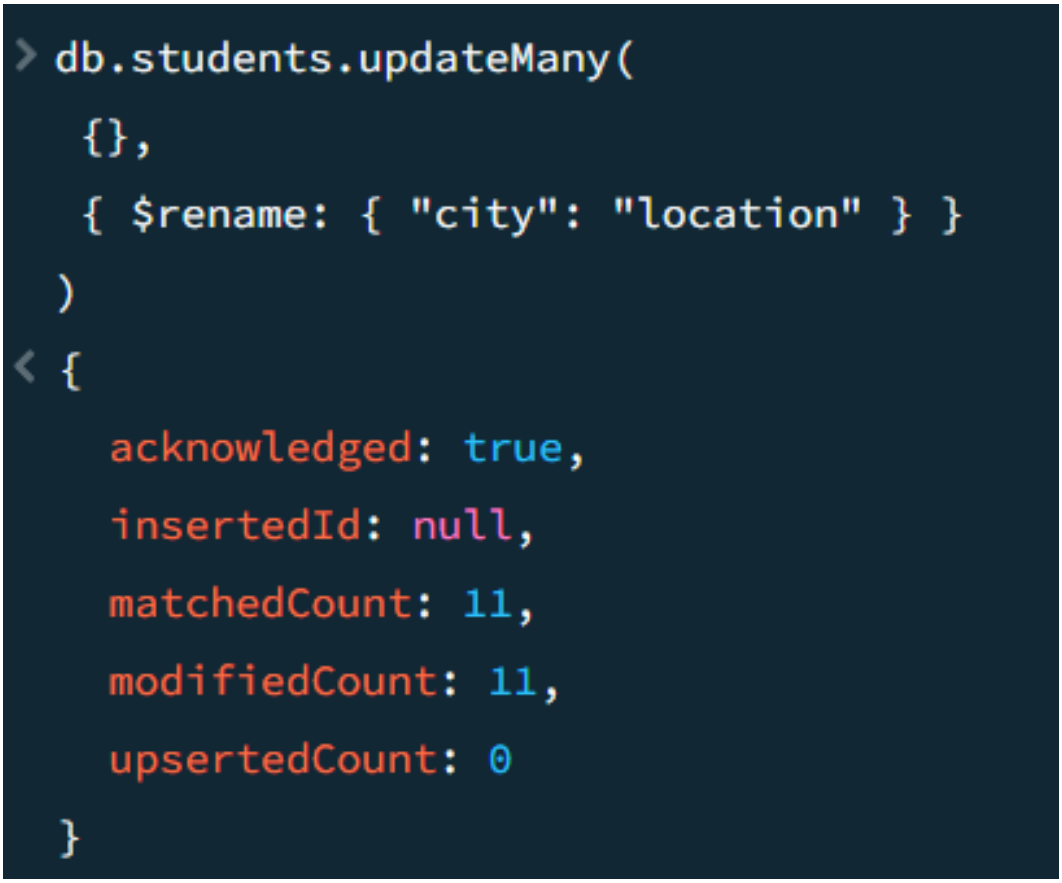
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 5,
  modifiedCount: 5,
  upsertedCount: 0
}

db.students.find({ department: "CSE" })
{
  _id: ObjectId('6a8b158489b6f0e955f4224b'),
  rollNo: 101,
  name: 'John',
  age: 20,
  department: 'CSE',
  marks: 92,
  city: 'Hyderabad',
  status: 'Active'
}
{
```

The updateMany() operation adds a 'status' field with value 'Active' to all CSE students. Response shows matchedCount: 5 and modifiedCount: 5, confirming all five CSE documents were updated.

Renaming a Field with $rename

```
db.students.updateMany({}, { $rename: { "city": "location" } })
```

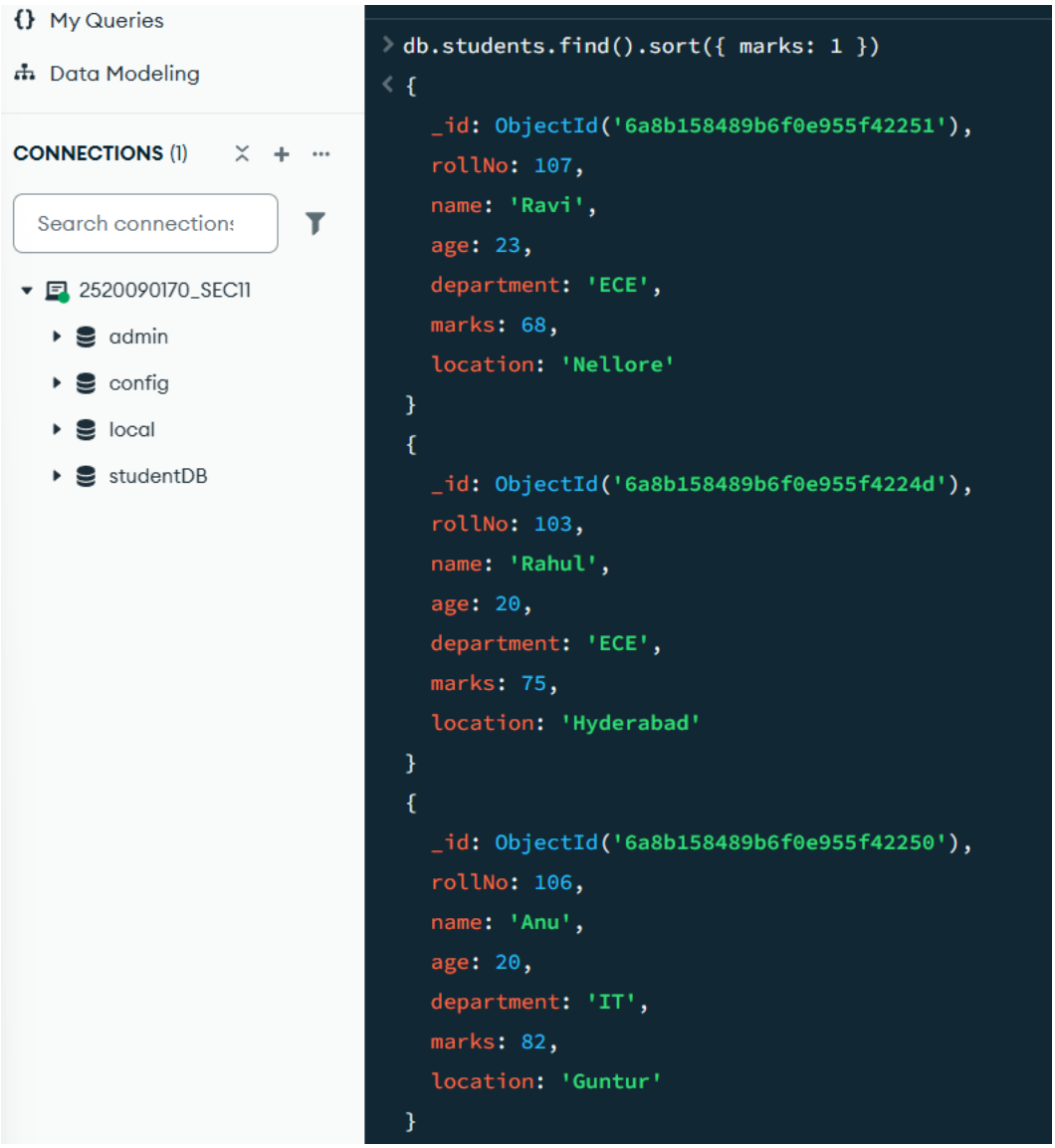


The $rename operator renames the 'city' field to 'location' across all 11 student documents. Response shows matchedCount: 11 and modifiedCount: 11, confirming the rename was applied to the entire collection.

6. Sorting and Limiting

Sorting Documents in Ascending Order

```
db.students.find().sort({ marks: 1 })
```

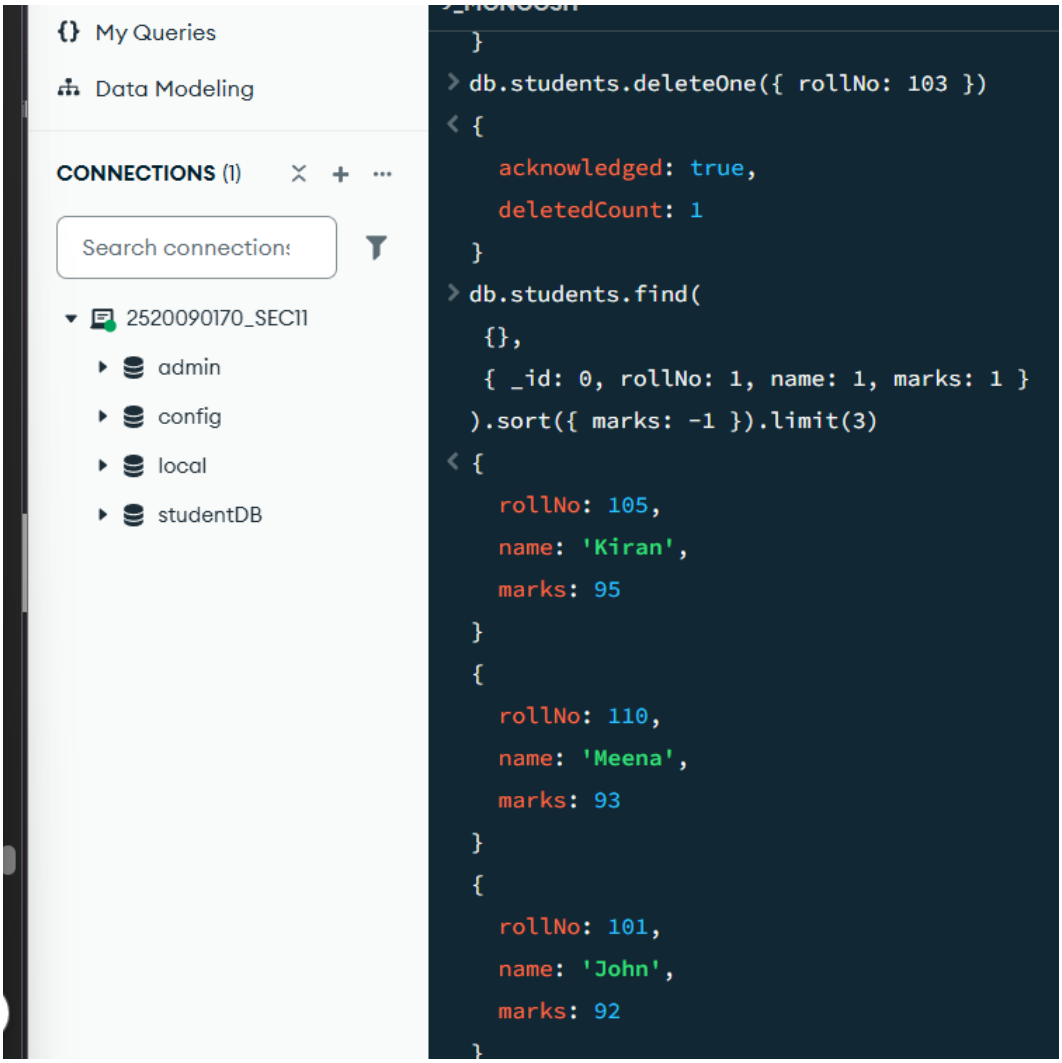


The sort() method with { marks: 1 } sorts all documents in ascending order of marks. Students are ordered starting from Ravi (68), Rahul (75), Anu (82), and continuing to higher marks.

Sorting and Limiting Results

```
db.students.find({}, { _id: 0, rollNo: 1, name: 1, marks: 1 })

  .sort({ marks: -1 }).limit(3)
```

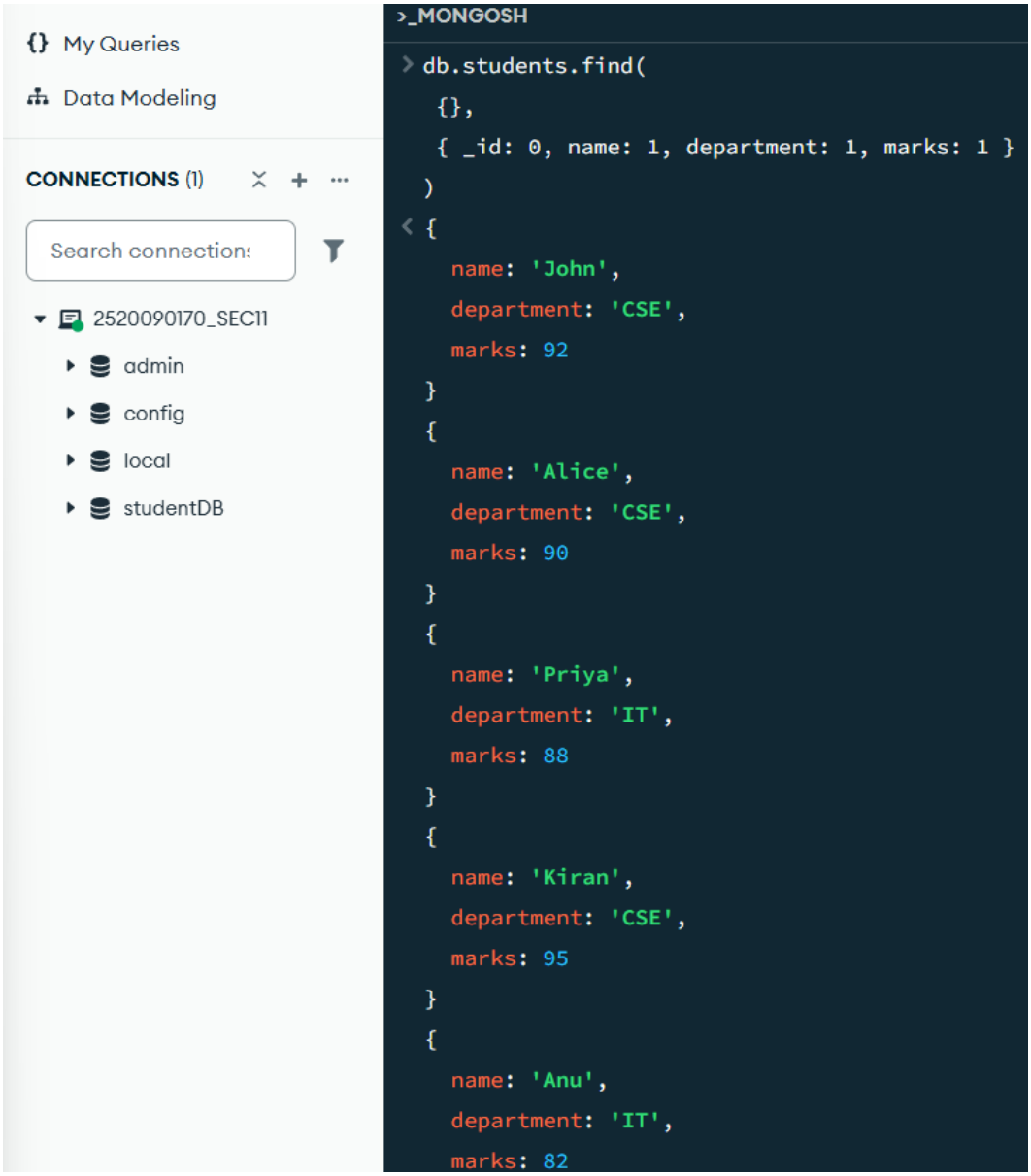


This query sorts students by marks in descending order (marks: -1) and limits to 3 documents. Projection displays only rollNo, name, and marks. Top 3 students: Kiran (95), Meena (93), John (92).

7. Delete Operations

Deleting a Single Document

```
db.students.deleteOne({ rollNo: 103 })
```



The deleteOne() method removes the student document with rollNo 103 (Rahul from ECE). The response confirms deletedCount: 1, indicating successful deletion.

8. Projection

Selecting Specific Fields

```
db.students.find({}, { _id: 0, name: 1, department: 1, marks: 1 })
```

```
> db.students.find({},{_id: 0, name: 1, department: 1,marks: 1 })
< {
    name: 'John',
    department: 'CSE',
    marks: 92
  }
  {
    name: 'Alice',
    department: 'CSE',
    marks: 90
  }
  {
    name: 'Priya',
    department: 'IT',
    marks: 88
  }
  {
    name: 'Kiran',
    department: 'CSE',
    marks: 95
  }
  {
    name: 'Anu',
    department: 'IT',
    marks: 82
  }
```

The projection { _id: 0, name: 1, department: 1, marks: 1 } selects only name, department, and marks fields while excluding _id. The output displays only the requested attributes for each document.

9. Indexing

Creating and Viewing Indexes

```
db.students.createIndex({ rollNo: 1 })
```

```
db.students.getIndexes()
```

```
db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' }
]
db.students.aggregate([
  { $match: { marks: { $gt: 80 } } }
])
{
  _id: ObjectId('6a8b158489b6f0e955f4224b'),
  rollNo: 101,
  name: 'John',
  age: 20,
  department: 'CSE',
  marks: 92,
  status: 'Active',
  location: 'Hyderabad'
}
{
  _id: ObjectId('6a8b158489b6f0e955f4224c'),
  rollNo: 102,
  name: 'Alice',
  age: 21,
  department: 'CSE',
  marks: 90,
  status: 'Active',
  location: 'Vijayawada'
}
```

The createIndex() method creates an ascending index on the rollNo field. The getIndexes() output displays all indexes: the default _id_ index and the newly created rollNo_1 index.

10. Aggregation Framework

Aggregation with $match Stage

```
db.students.aggregate([

  { $match: { marks: { $gt: 80 } } }

])
```

```
db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { rollNo: 1 }, name: 'rollNo_1' }
]
db.students.aggregate([
  { $match: { marks: { $gt: 80 } } }
])
{
  _id: ObjectId('6a8b158489b6f0e955f4224b'),
  rollNo: 101,
  name: 'John',
  age: 20,
  department: 'CSE',
  marks: 92,
  status: 'Active',
  location: 'Hyderabad'
}
{
  _id: ObjectId('6a8b158489b6f0e955f4224c'),
  rollNo: 102,
  name: 'Alice',
  age: 21,
  department: 'CSE',
  marks: 90,
  status: 'Active',
  location: 'Vijayawada'
}
```

The $match stage filters documents to return only students with marks > 80. This is the first pipeline stage and acts similarly to the find() query filter.

Grouping and Counting with $group and $sum

```
db.students.aggregate([

  { $group: { _id: "$department", totalStudents: { $sum: 1 } } }

])
```

```
> db.students.aggregate([
    {
      $group: {
        _id: "$department",
        totalStudents: { $sum: 1 }
      }
    }
  ])
< {
    _id: 'AI&DS',
    totalStudents: 2
  }
  {
    _id: 'IT',
    totalStudents: 2
  }
  {
    _id: 'CSE',
    totalStudents: 5
  }
  {
    _id: 'ECE',
```

The $group stage groups documents by department and uses $sum: 1 to count students in each group. Results show: AI&DS (2), IT (2), CSE (5), ECE (1) – totaling 10 documents after deletion of Rahul.

Finding Maximum Marks with $max

```
db.students.aggregate([

  { $group: { _id: null, highestMarks: { $max: "$marks" } } }

])
```



The $max operator within $group calculates the highest marks across all students. Result shows highestMarks: 95, which belongs to Kiran.

11. Multi-Stage Aggregation Pipeline

Complex Aggregation with $sort, $limit, and $project

```
db.students.aggregate([

  { $sort: { marks: -1 } },

  { $limit: 3 },

  { $project: { _id: 0, rollNo: 1, name: 1, marks: 1 } }

])
```

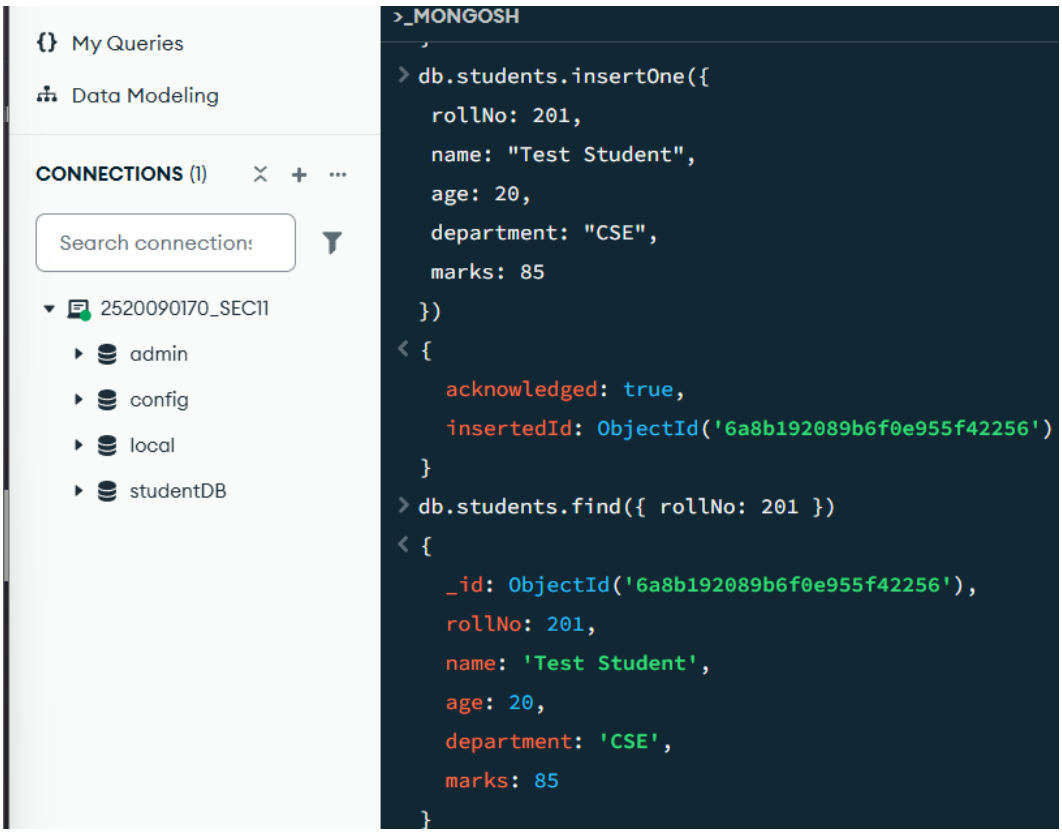
```
> db.students.aggregate([
    { $sort: { marks: -1 } },
    { $limit: 3 },
    { $project: { _id: 0, rollNo: 1, name: 1, marks: 1 } }
  ])
< {
    rollNo: 105,
    name: 'Kiran',
    marks: 95
  }
  {
    rollNo: 110,
    name: 'Meena',
    marks: 93
  }
  {
    rollNo: 101,
    name: 'John',
    marks: 92
```

This multi-stage pipeline sorts documents by marks descending, limits to 3 results, and projects only rollNo, name, and marks. Output shows: Kiran (95), Meena (93), John (92).

12. Complete CRUD Operation Sequence

Create: insertOne() – Adding a New Student

```
db.students.insertOne({ rollNo: 201, name: "Test Student",
  age: 20, department: "CSE", marks: 85 })
```



A new student with rollNo 201 is inserted into the collection. Response confirms successful insertion with acknowledged: true and the inserted document ID.

Read: find() – Retrieving the Inserted Document

```
db.students.find({ rollNo: 201 })
```

The find() query retrieves the newly inserted student with all fields including the MongoDB-generated _id. Document exists in collection with all attributes preserved.

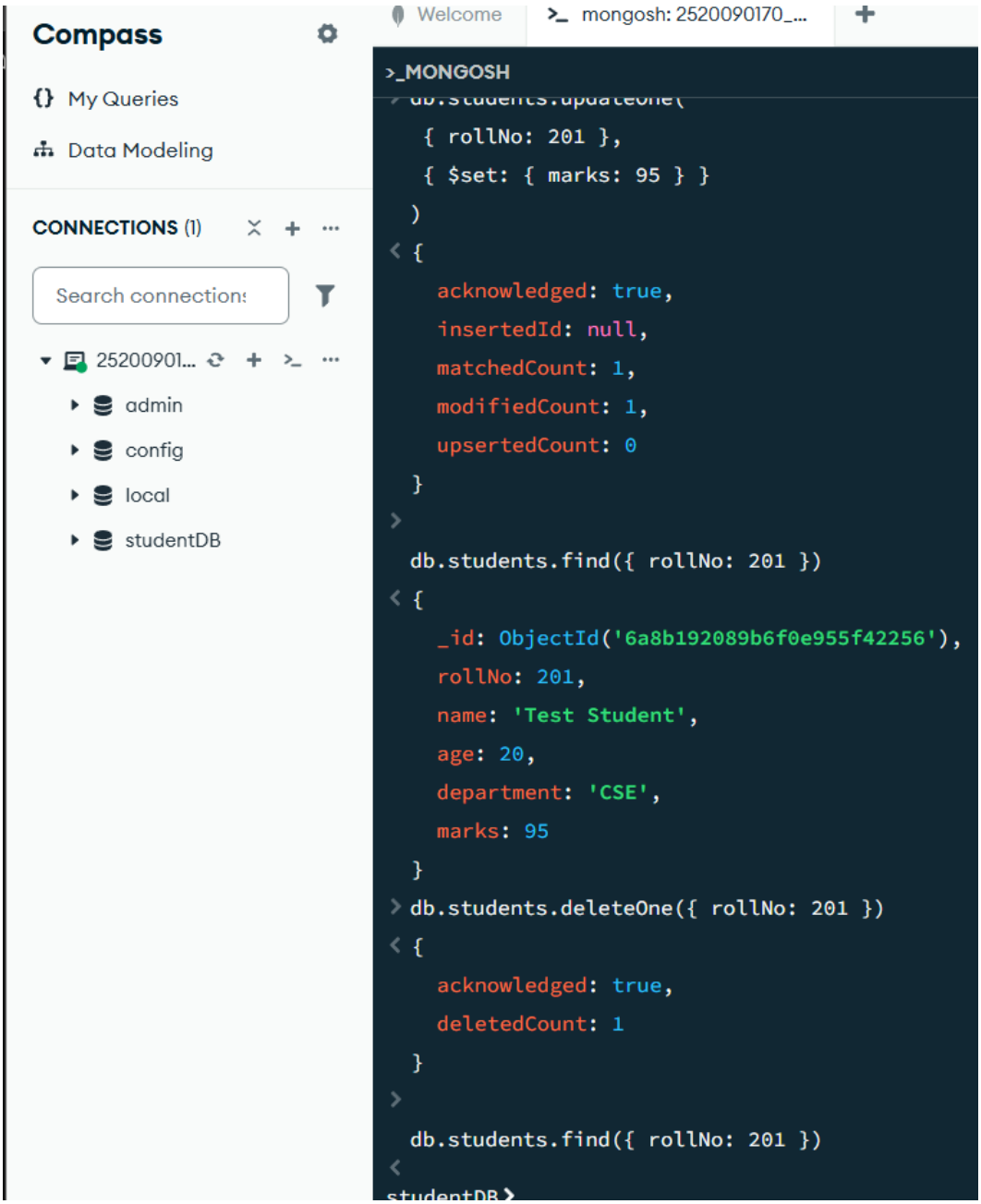
Update: updateOne() – Modifying the Document

```
db.students.updateOne({ rollNo: 201 }, { $set: { marks: 95 } })
```

The updateOne() operation with $set changes marks from 85 to 95 for rollNo 201. Response confirms matchedCount: 1 and modifiedCount: 1.

Verify Update: find() – Confirming the Modification

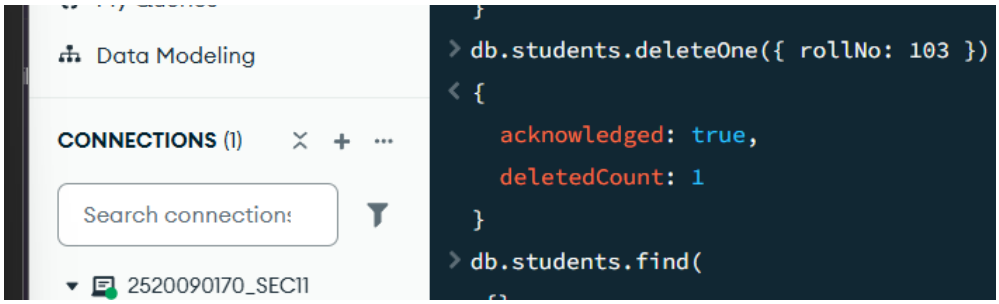
```
db.students.find({ rollNo: 201 })
```



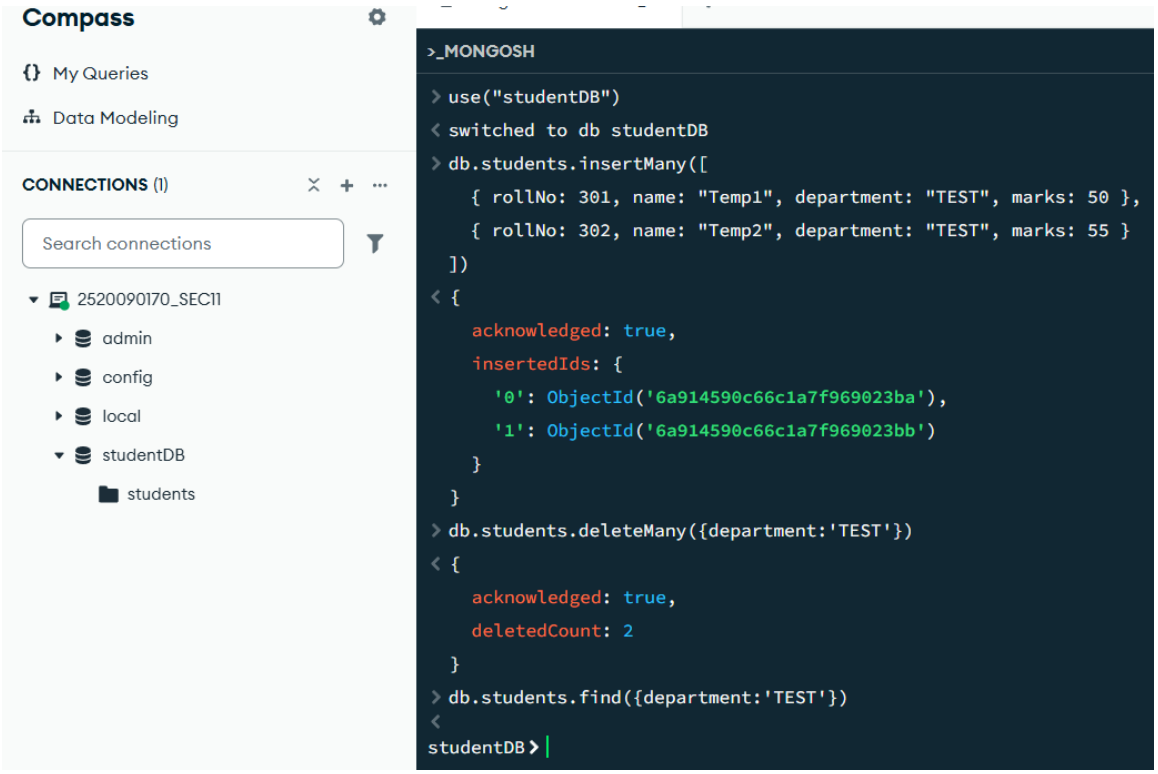
The find() query confirms marks have been successfully updated to 95 for rollNo 201.

Delete: deleteOne() – Removing the Document

```
db.students.deleteOne({ rollNo: 201 })
```



The deleteOne() method removes the student document with rollNo 201. Response confirms deletedCount: 1, indicating successful deletion.
The deleteMany() method removes all the records that satisfy the condition with department name TEST . Response confirms deletedCount: 2, indicating successful deletion.



Verify Deletion: find() – Confirming Document Removal

```
db.students.find({ rollNo: 201 })
```

The final find() query returns an empty result (no output), confirming the student with rollNo 201 and the students with the department name TEST no longer exists in the collection after deletion.

Result

All MongoDB commands for database and collection creation, CRUD operations (Create, Read, Update, Delete), comparison operators ($gt, $gte, $ne), logical operators ($and), field renaming with $rename, sorting (ascending and descending), limiting, projection, indexing, distinct values, and aggregation framework operations (including $match, $group, $sum, $max, $sort, $limit, and $project) were executed successfully on the student collection using MongoDB Compass and mongosh. The practical demonstrates complete functionality of MongoDB operations on relational student data with comprehensive examples for all major command categories.